INPUT E OUTPUT FILE

Per registrare grandi sequenze di dati sin ora vi è stato l'utilizzo delle strutture, esse però dispongono di una memoria **temporanea**, la quale resta attiva solo nella fase d'esecuzione del programma. Per memorizzare i dati in maniera **permanente** sulla memoria del PC si utilizzano i **file**.
L'accesso a questi ultimi si divide, a loro volta per gestire i dati, in **accesso sequenziale** e **accesso casuale**.
Il C interpreta i file come un **flusso sequenziale di byte**, ogni file termina con un marcatore denominato **end-of-file**. L'apertura di un file viene definita come un atto di **stream**, all'esecuzione di un programma possiamo avere 3 diversi stream:

- **STANDARD INPUT**
- **STANDARD OUTPUT**
- **STANDARD ERROR**

Lo stream fornisce la **comunicazione** tra file e programma. Infatti attraverso gli stream di input possiamo leggere i dati inseriti da tastiera, e visualizzare a schermo questi dati attraverso lo stream di output.
Quando si apre un file, il programma riceve un puntatore ad una struttura **FILE** che contiene le informazioni usate per elaborare i file. Queste funzioni sono presenti nella libreria `<stdio.h>`; i tre standard precedenti vengono manipolati corrispettivamente usando **stdin**, **stdout** e **stderr**.

La funzione utilizzata per **leggere** un carattere da un file è ***fgetc***, infatti la sua chiamata `fgetc(stdin)` legge un carattere dallo standard di input. Il lavoro svolto da questa funzione è pressoché identico alla funzione **getchar()**.
La funzione utilizzata per **inserire** un carattere su un file è ***fputc***, diversamente dalla lettura questa funzione riceve come argomenti il carattere *'a'* da inserire sul file e il puntatore per il file su cui dovrà scrivere tale carattere; la chiamata della sua funzione si esprime in questo modo `fputc('a',stdout)`, ovvero il carattere *'a'* viene scritto sullo standard di output.
Esistono numerose funzioni che si comportano in maniera analoga e con nomi simili, per esempio le funzioni ***fgets*** e ***fputs***, come nelle stringhe, possono leggere/scrivere un intera **riga** su un file.

CREAZIONE FILE CON ACCESSO SEQUENZIALE

Il linguaggio C **non impone una struttura definita** per i file, è a discrezione del programmatore definire una struttura logica idonea per i dati scritti e letti da un file.
Nei file ad accesso sequenziale i dati vengono letti/scritti **uno dopo l'altro**. Questi file trovano utilizzo nelle applicazioni in cui si gestiscono informazioni ordinate (come una lista).
Per simulare un **record**, il file viene progettato in modo che ogni riga sia **logicamente**

correlata con quella successiva (come le informazioni di un cliente). Un campo del record successivamente verrà scelto per rappresentare la **chiave del record**, ovvero un campo che identifica **univocamente** ogni record (come l'ID del conto). Per gestire meglio il flusso dei dati è buona prassi che i record siano memorizzati nell'ordine definito dalla chiave.
Al termine della creazione del file, quest'ultimo può essere usato da **persone esterne o moduli** per eseguire determinate operazioni, come:

- Recupero dati
- Analisi
- Aggiornamenti

PUNTATORE A UN FILE

```
1  // Fig. 11.2: fig11_02.c
2  // Creazione di un file sequenziale
3  #include <stdio.h>
4
5  int main(void)
6  {
7     FILE *cfPtr; // cfPtr = puntatore al file clients.txt
8
9     // fopen apre il file per la scrittura
10    if ((cfPtr = fopen("clients.txt", "w")) == NULL) {
11       puts("File could not be opened");
12    }
13    else {
14       puts("Enter the account, name, and balance.");
15       puts("Enter EOF to end input.");
16       printf("%s", "? ");
17
18       unsigned int account; // numero del conto
19       char name[30]; // nome del titolare del conto
20       double balance; // saldo del conto
21
22       scanf("%d%29s%lf", &account, name, &balance);
23
24       // scrivi numero del conto, nome e saldo nel file con fprintf
25       while (!feof(stdin)  ) {
26          fprintf(cfPtr, "%d %s %.2f\n", account, name, balance);
27          printf("%s", "? ");
28          scanf("%d%29s%lf", &account, name, &balance);
29       }
30
31       fclose(cfPtr); // fclose chiude il file
32    }
33 }
```

```
Enter the account, name, and balance.
Enter EOF to end input.
? 100 Jones 24.98
? 200 Doe 345.67
? 300 White 0.00
? 400 Stone -42.16
? 500 Rich 224.62
? ^Z
```

Analizzando questo codice possiamo notare nella *riga 7* l'utilizzo di un puntatore (**cfPtr**). Questo puntatore punta alla struttura FILE ed in C ogni file aperto deve possedere il suo puntatore di tipo *FILE* dichiarato separatamente, per riferirsi ad un tale file.

Analizzando sempre il codice in sovraimpressione analizziamo nella *riga 10* il **nome del file** con cui il programma deve **stabilire una comunicazione**, la connessione sarà attivata grazie all'uso della funzione ***fopen***. La funzione richiede due parametri, il **nome** dettato precedentemente per instaurare la connessione e la **modalità di apertura**, in questo caso ***w*** indica l'apertura a scrittura. Se il file non è già esistente viene **creato**. Nel nostro codice si controlla se il puntatore al file è **NULL** per determinare se il file esista o meno.
!ATTENZIONE! l'apertura in scrittura di un file già esistente, ne comporta la **cancellazione** di tutto ciò che era memorizzato precedentemente sopra.

Analizziamo una nuova funzione utilizzata nel codice, $feof$. Essa è presente nella *riga 25* ed il suo utilizzo è **controllare** se l'indicatore di end-of-file (**EOF**) è impostato per lo standard di input. La funzione **restituisce un qualsiasi valore diverso da 0** quando è stato impostato l'indicatore di termine.
L'inserimento dell'**indicatore** infatti determina che non ci sono più dati da inserire nel file, portando a **termine** l'inserimento su file. Infatti questa combinazione di tasti è gestita dagli standard di input stessi, combinazione che varia a seconda del Sistema Operativo (nel nostro caso, come si evince dal terminale, utilizziamo $Ctrl - z$ poiché lavoriamo su **Windows**).

Spostiamoci ad una delle ultime righe del file, la numero *31*, ove nella quale è presente la funzione di **chiusura** del file, ***fclose***. Questa funzione riceve il **puntatore** del file e non il suo nome, poiché la connessione è stabilita tramite il puntatore e tramite quest'ultimo possiamo chiuderla.
La funzione $fclose$ può essere omessa ed il SO gestirà autonomamente la chiusura del file al termine dell'esecuzione del programma, ma è di estrema **importanza** chiamare la funzione $fclose(puntatoreFILE)$ per chiudere il file poiché, in un codice complesso ad esempio, un file aperto che però non necessita più della nostra attenzione trattiene **risorse** fondamentali di cui altri programmi sono in **attesa**, rallentando la macchina.

MODALITA' DI APERTURA DEI FILE

Come abbiamo visualizzato, precedentemente abbiamo aperto un file in modalità scrittura passando il parametro ***w*** alla funzione $fopen$. Esistono altre **modalità** per aprire un file:

Modalità	Descrizione
r	Apre un file esistente per la lettura.
w	Crea un file per la scrittura. Se il file esiste già, elimina i contenuti correnti.
a	Apre o crea un file per scrivere alla fine del file – cioè, le operazioni di scrittura aggiungono dati al file.
r+	Apre un file esistente per l'aggiornamento (lettura e scrittura).
w+	Crea un file per l'aggiornamento. Se il file esiste già, elimina i contenuti correnti.
a+	Append: apre o crea un file per l'aggiornamento; tutta la scrittura è effettuata alla fine del file – cioè, le operazioni di scrittura aggiungono dati al file.
rb	Apre un file esistente per la lettura in forma binaria.
wb	Crea un file per la scrittura in forma binaria. Se il file esiste già, elimina i contenuti correnti.
ab	Append: apre o crea un file per la scrittura alla fine del file in forma binaria.
rb+	Apre un file esistente per l'aggiornamento (lettura e scrittura) in forma binaria.

Modalità	Descrizione
wb+	Crea un file per l'aggiornamento in forma binaria. Se il file esiste già, elimina i contenuti correnti.
ab+	Append: apre o crea un file per l'aggiornamento in forma binaria; la scrittura è effettuata alla fine del file.

Inoltre tutte le modalità che seguono una ***b*** finale sono utilizzate per la **manipolazione di file binari**, generalmente usate per l'accesso ai file in modalità **casuale**.

ERRORI TIPICI NELL'APERTURA DI UN FILE

- Aprire un file inesistente in modalità lettura
- Aprire un file *r/w* senza avere i diritti d'accesso
- Aprire un file in scrittura quando lo spazio di sistema è esaurito
- Aprire un file in modalità scrittura quando si vuole aggiornarlo (*r+*), comporta all'eliminazione di tutti i dati presenti nel file

LETTURA DA UN FILE CON ACCESSO SEQUENZIALE

Dopo aver creato e inserito dei dati su un file bisogna considerare l'opzione contraria, cioè la possibilità di leggere dei dati da un file per usarli nel codice. Useremo il file *"clienti.txt"* creato in precedenza per analizzare il processo di lettura, leggendo tutti i dati inseriti prima.

```
5  int main(void)
6  {
7     FILE *cfPtr; // cfPtr = puntatore al file clients.txt
8
9     // fopen apre il file per la lettura
10    if ((cfPtr = fopen("clients.txt", "r")) == NULL) {
11       puts("File could not be opened");
12    }
13    else { // leggi account, name e balance dal file
14       unsigned int account; // numero del conto
15       char name[30]; // nome del titolare del conto
16       double balance; // saldo del conto
17
18       printf("%-10s%-13s%s\n", "Account", "Name", "Balance");
19       fscanf(cfPtr, "%d%29s%lf", &account, name, &balance);
20
```

```
21       // finché non si incontra un end of file
22       while (!feof(cfPtr)  ) {
23          printf("%-10d%-13s%7.2f\n", account, name, balance);
24          fscanf(cfPtr, "%d%29s%lf", &account, name, &balance);
25       }
26
27       fclose(cfPtr); // fclose chiude il file
28    }
29 }
```

```
Account   Name         Balance
100       Jones          24.98
200       Doe           345.67
300       White           0.00
400       Stone         -42.16
500       Rich          224.62
```

Analizziamo la *riga 7* in cui creiamo un puntatore a FILE, come in precedenza. Dopo di che ci spostiamo alla *riga 10*, in cui chiediamo la possibilità di creare una connessione in lettura del file e verificare se il file è stato aperto con successo o meno. Definiamo le variabili che andremo a leggere tramite la funzione $fscanf$. Usiamo $fscanf$ in quanto è progettato per leggere dai file, poiché richiede come argomento il puntatore ad esso; azione impossibile da eseguire da una normale $scanf$ perché limitata alla lettura di *stdin*, la quale libreria non gestisce i flussi di dati (come i file).
Tramite ciclo andremo a leggere tutti gli altri dati fin quando non saranno esauriti.
Come eseguito per la scrittura, utilizziamo anche qui la funzione $feof(puntatoreFILE)$, la quale restituirà *0* solo dopo aver letto dei dati **inesistenti**.

Anche qui, come per la scrittura, è fondamentale **chiudere** il file dopo l'utilizzo, per i motivi spiegati precedentemente.

CONFUSIONE TRA I NOMI

Nella programmazione classica, usiamo la *scanf* per inserire dei dati dall'esterno all'interno del codice e la *printf* per leggere dei dati dal codice e presentarli all'esterno. Ciò non deve portare però a confondere l'azione di scrittura e lettura nei file; poiché usiamo la *fprintf* per scrivere sui file non vuol dire che essa sia un operazione di lettura ma è un operazione di **invio** dei dati ad un altra zona. Analogicamente usiamo la *fscanf* non per scrivere i dati ma per **estrarre** dati ed inserirli in altre variabili.
Anche se i nomi possono sembrare forvianti rappresentano perfettamente il loro scopo nel programma, tutto ciò è dimostrato proprio dal suffisso *f* presente all'inizio delle due funzioni:

- La *fscanf* estrae informazioni per inserirle in altri luoghi
- La *fprintf* mostra i dati presenti nel codice al file, ecco perché è un inserimento
- La *scanf* importa dati da tastiera al codice
- La *printf* porta dati dal codice ad un altra periferica esterna al codice, ecco perché è un output, una fuoriuscita di dati presso un altra zona.

REIMPOSTARE IL PUNTATORE

In alcuni codici è richiesto che i dati vengano letti in maniera **sequenziale** più di una volta. Se volessimo ripartire dall'inizio durante l'esecuzione del nostro programma necessitiamo che il puntatore al file riparta dal **byte 0**, per risolvere ciò si utilizza la funzione $rewind(puntatoreFILE)$, la quale riposiziona all'inizio il **puntatore di posizione del file**.
Il nome *"Puntatore di posizione"* è solo a scopo teorico, poiché il valore della variabile di posizione è un **intero** e non un puntatore, esso rappresenta, nel file oggetto, il byte della prossima istruzione di tipo o di lettura o di scrittura. Il nome tecnico di questa valore è detto **file offset**.

FILE BINARI

I file binari sono denominati anche **file ad accesso casuale**, poiché il puntatore non scorre sequenzialmente i contenuti, di fatto l'accesso avviene puntando ad una specifica locazione di memoria.
I file binari garantiscono una maggiore **flessibilità**, i contenuti possono essere aggiornati senza **sovrascrivere** i dati memorizzati precedentemente.

FILE BINARI AD ACCESSO CASUALE

I dati in un file ad accesso casuale **non sono formattati**, sono memorizzati come *raw bytes*, tutti i tipi di dati utilizzano la stessa quantità di memoria, come tutti i record hanno la stessa lunghezza fissa, in più il formato dei dati non sono **Human readable** ovvero usano un linguaggio difficile e non facilmente comprensibile alla lettura di un essere umano.

LETTURA E SCRITTURA IN BLOCCHI DI BYTE

Esistono varie funzioni che ci permettono di scrivere su blocchi di byte o anche di dati testuali; queste funzioni trovano utilizzo specialmente nella scrittura di un intera struct su un file.

- $intfread$*(void *punt,dimBlocco,numBlocchi, FILE *fp)*: Legge dal file *fp* un numero di byte pari alla **dimensione dei blocchi * numero dei blocchi** e li memorizza nell'area di memoria puntata dal **puntatore**. Restituendo in fine il numero di blocchi letti.
- $intfwrite$*(void *punt, dimBlocco, numBlocchi, FILE *fp)*: Esegue la stessa operazione precedente ma in **scrittura**.

FSEEK

La funzione $FSEEK$ ci permette di accedere a degli specifici byte del file, come se quest'ultimo fosse un array di blocchi di byte.
La funzione:*int fseek(FILE *fp, long offset, int refpoint)*, imposta la posizione corrente ad un valore pari ad uno spostamento del valore dell'**offset**, il quale viene calcolato in base a dei punti di partenza:

- All'inizio del file se refpoint vale SEEK_SET che è un valore costante impostato dalla libreria *stdio.h*
- L'attuale posizione corrente, la quale è un altra costante della medesima libreria
- La fine del file se refpoint vale SEEK_END
  Il valore restituito dall'offset è 0 se l'operazione è avvenuta con successo altrimenti ha un valore differente.

REWIND

```
void rewind(FILE *fp)
```

La funzione **rewind** è utile per *riavvolgere* la posizione corrente del file senza doverlo necessariamente chiudere e riaprirlo. Per esempio se volessimo tornare a contare il file dall'inizio possiamo avvalerci di questa funzione.
Tra la chiamata della funzione $rewind$ e la chiamata alla funzione di *fseek(fp,0,SEEK_SET)*, non vi è alcuna differenza, entrambe svolgono la stessa funzionalità ma è buona pressi utilizzare rewind poiché rende il codice più leggero e aumenta la sua **leggibilità**, dato che non dovremmo specificare tutti i parametri come nella funzione $fseek$.